as you've already seen; another case is where the ownership of the lock needs to be transferred from one scope to another.

3.2.7. Transferring mutex ownership between scopes

Because `std::unique_lock` instances don't have to own their associated mutexes, the ownership of a mutex can be transferred between instances by *moving* the instances around. In some cases such transfer is automatic, such as when returning an instance from a function, and in other cases you have to do it explicitly by calling `std::move()`. Fundamentally this depends on whether the source is an *lvalue*—a real variable or reference to one—or an *rvalue*—a temporary of some kind. Ownership transfer is automatic if the source is an rvalue and must be done explicitly for an lvalue in order to avoid accidentally transferring ownership away from a variable. `std::unique_lock` is an example of a type that's *movable* but not *copyable*. See appendix A, section A.1.1, for more about move semantics.

One possible use is to allow a function to lock a mutex and transfer ownership of that lock to the caller, so the caller can then perform additional actions under the protection of the same lock. The following code snippet shows an example of this: the function `get_lock()` locks the mutex and then prepares the data before returning the lock to the caller:

```
std::unique_lock<std::mutex> get_lock()
{
    extern std::mutex some_mutex;
    std::unique_lock<std::mutex> lk(some_mutex);
    prepare_data();
    return lk;          <-(1)
}
void process_data()
{
    std::unique_lock<std::mutex> lk(get_lock());      <-(2)
    do_something();
}
```

Because `lk` is an automatic variable declared within the function, it can be returned directly (1) without a call to `std:move()`; the compiler takes